

Two Kinds of Hallucination, One Positive Class: A Concentrated Conflation in RAGTruth

Hugo Gómez García 

Universidad de Oviedo (student)

gomezgarciahu@gmail.com

ORCID: [0009-0003-4137-1705](https://orcid.org/0009-0003-4137-1705)

Code and data: <https://github.com/hugoomez/rag-hallucination-detector>

Abstract

Every RAGTruth (Niu et al., 2024) evaluation whose preprocessing is publicly inspectable scores its gold hallucination class as a single, undifferentiated positive: a span is hallucinated or it is not. The benchmark’s own annotators mark a distinction that both such evaluations discard. The `implicit_true` field records spans that are unsupported by the retrieved context but happen to be true — 13.5% of gold spans by count, 14.56% by character mass — and marks them, in 90.6% of cases, with the annotators’ own low-intensity severity prefix, against 4.4% for other gold spans. These two subclasses are marked as differing in severity by the benchmark’s own annotators, and are nonetheless scored identically. The conflation is not a scattering of annotator disagreement: it concentrates in one span type — 73.64% of spans typed “Subtle Baseless Info” are themselves the low-intensity, ungrounded-but-true subclass — and is unevenly distributed by task type, the structure of a real, recognized subclass rather than of noise. We quantify what this structured conflation costs under RAGTruth’s official scoring: because the protocol counts every flagged character directly, any prediction set that excludes the low-intensity subclass is capped at 90.5% char-overlap span recall against gold, independent of any model — a direct consequence of the conflation, not a second, independent result. We also tested whether the conflation could be exploited at training time, by down-weighting the low-intensity subclass in the loss under a pre-registered decision rule; across three lambda values it produced a monotonic, dose-dependent decline in clean-span and response F1 rather than an improvement, ruling out both “the model is insensitive to this distinction” and “the intervention was too weak to register” as explanations — a negative result we report as one rather than repackage as a further finding. We report this as a benchmark-quality contribution, not a detection method, and show that the metric needed to distinguish these subclasses already exists in data the benchmark ships but no published evaluation uses.

Keywords: RAGTruth; hallucination detection; retrieval-augmented generation; benchmark evaluation; label-class conflation; annotation metadata

1. Introduction

Every RAGTruth (Niu et al., 2024) evaluation whose preprocessing we could inspect treats the benchmark’s gold annotations as a single, undifferentiated positive class. A span is hallucinated or it is not; a response is hallucinated or it is not; the reported F1 aggregates over that binary. This is how the benchmark’s own reference baseline scores itself, how LettuceDetect (Kovács & Recski, 2025) scores itself, and how the comparison tables in this literature are assembled from

figures whose underlying preprocessing is, in most cases, unverifiable (§2.4).

That positive class is not homogeneous. RAGTruth annotates under a faithfulness objective — it targets claims that are *unsupported or contradictory* with respect to the retrieved context — and under that objective a span can earn a positive label in two materially different ways. It can assert something the context does not support and that is also false: **ungrounded-and-false**. Or it can assert something the context does not support but which happens to be true: **ungrounded-but-true**. Both are hallucinations under the stated objective, and RAGTruth labels both positive, correctly. These two subclasses are marked as differing in severity by the benchmark’s own annotators, and are nonetheless scored identically.

The benchmark ships the field that separates them. `implicit_true`, defined in the corpus README as marking a span that “is correct while the info is not mentioned in the context,” is set on 1,928 of 14,289 gold spans — 13.5% by span count, and 14.56% of gold-span character mass. The two figures are different quantities and are reported here as such; an earlier version of this work conflated them.

This yields the gap the paper addresses. **Published RAGTruth evaluations score ungrounded-and-false content and ungrounded-but-true content identically, even though the benchmark’s own annotators mark them as differing in severity — so no reported F1 distinguishes performance on the two subclasses.**

The distinction matters most where a detector’s output feeds a human reviewer rather than an automated block: such a deployment would reasonably want to prioritize ungrounded-and-false content — the subclass a reviewer must catch — over ungrounded-but-true content, which is wrong about grounding but not about the world. We do not evidence how production RAG deployments actually triage flagged output, and the point below does not depend on that evidence. It is narrower: RAGTruth’s official scoring cannot express such a priority even where a deployment holds it. The protocol counts a decision to deprioritize the low-severity subclass as an equivalent miss to failing on the high-severity one, so a detector built for triage is scored by the same metric as one that draws no such distinction. No published evaluation on this benchmark can say whether a system was built with this priority in mind or simply happened to miss fewer ungrounded-but-true spans.

A reader may reasonably respond that every benchmark carries some label noise, so what? The objection does not apply here, for two reasons, and it is worth settling before the evidence rather than after it. First, nothing in this paper is *estimated* noise. `implicit_true` is a self-declared, machine-readable field that the benchmark publishes with the data, and that the systems trained on that data discard: we verified in code (2026-07-25) that LettuceDetect’s `preprocess_ragtruth.py` reads only `start`, `end`, and `label_type`, and that RAGTruth’s own vendored reference baseline contains no occurrence of the field at all. Luna (Belyi et al., 2024/2025) and RAG-HAT (Song et al., 2024) publish neither code nor weights; their treatment of the field is unverified and we do not assume it. Second, and more importantly, the field does not mark an error. Two distinct signals in the released data establish that it is a **severity qualifier applied within the positive class**, not a retraction of the label. The README’s own wording separates correctness from groundedness. And RAGTruth’s per-span meta field — a single annotator-written string, whose first line is a severity prefix and whose remainder is free text — carries both halves of the same signal: it consistently asserts that the content is true but ungrounded (“these details are correct, however, not directly mentioned in the passages”), and its severity prefix opens with `LOW` for 90.6% of flagged spans, against 4.4% of all other gold spans. The labels are right. What is at issue is that two error types of very different consequence are being scored as one. We therefore use the term **label-class conflation** throughout, and not “label noise” or “annotator disagreement.”

The magnitude is exact, because the RAGTruth test set is the object of study rather than a sample from which anything is estimated. Flagged spans account for 8,151 of 85,877 characters of test gold span mass — 9.49%. Stated as what it actually bounds: **under RAGTruth’s official scoring, any prediction set that omits the low-intensity subclass from its positives is capped at 90.5% char-overlap span recall against gold** — a constraint on the scoring protocol, not a claim about what any detector does. This is a bound on achieving two goals at once, not a ceiling on a metric in isolation — a system that flags everything still reaches 100% recall — and every statement of it in this paper carries that conditional. At the coarser response level the same argument gives a corroborating instance: 49 of the 943 hallucinated test responses consist *entirely* of flagged spans, so the same protocol-level argument caps response-level recall at 94.8% for any prediction set that omits the low-intensity subclass.

One clarification matters before the paper puts this bound to use. It is a property of RAGTruth’s scoring protocol — computable directly from the released gold labels, independent of whether any detector can, or should, restrict its predictions to the high-severity subclass. It does not propose such a detector, and no experiment in this paper builds one. What it shows is that any prediction set scored against official gold while omitting the low-intensity subclass is capped at that recall by construction, before any model exists.

§4 establishes what this property of the benchmark actually is: not a scattering of annotator disagreement but a **structured subclass** — concentrated in one span type, unevenly distributed by task, and already marked as low-severity by the annotators who labelled it. That structure is this paper’s central finding. Two things follow from it, and neither is inflated into a co-equal second finding. §6 quantifies what the conflation costs under RAGTruth’s official scoring — a direct arithmetic consequence of §4, computable before any model is trained: the scoring protocol counts every flagged character directly, undiluted, so it fixes what any reported RAGTruth recall figure can mean once the low-intensity subclass is excluded from what counts as caught. §5 asks whether the same conflation can instead be *exploited* at training time, by teaching a model to discount the low-severity subclass through loss down-weighting; under a pre-registered decision rule, across three lambda values, it never did — a monotonic, dose-dependent decline instead of an improvement, ruling out both “the model can’t tell” and “the intervention was too weak” as explanations. The two are worth holding side by side for one reason, developed in §7.1 as a discussion point rather than asserted here as the thesis: the same subclass is large enough, counted undiluted, to bend a scoring function, and small enough, diluted across a full training objective, that removing it measurably costs rather than helps — a difference in the size of the quantity each stage operates over, not a contradiction between the two results.

This is a benchmark-quality contribution, not a detection method. No number reported here is a new record, and none needs to be. The token-level detector used as the instrument throughout — a ModernBERT-based (Warner et al., 2024/2025) binary token classifier replicating LettuceDetect’s recipe — is prior, separately developed infrastructure of this project, documented in the repository (ADR-013, ADR-014, ADR-020) and not re-argued here.

2. Background and Related Work

2.1 RAGTruth’s construction and faithfulness objective

RAGTruth (Niu et al., 2024) is a hallucination corpus of 17,790 LLM responses spanning three retrieval-augmented task types — question answering, summarization, and data-to-text generation — with human annotation at the word/span level. Its annotation target is **faithfulness, not factuality**: annotators mark claims that are *unsupported or contradictory* with respect to the retrieved context. A claim that is true of the world but absent from the context is therefore in scope, by design. Spans carry a `label_type` drawn from four categories along two axes, in-

tensity (Evident/Subtle) and kind (Baseless Info/Conflict), and a free-text meta comment. The official split is 15,090 train responses and 2,700 test responses, of which 943 are hallucinated and 1,757 faithful; all test figures in this paper are on that official test split. Span-level scoring in the token-level detector literature is character-overlap micro-F1, and response-level scoring derives a response label from its token predictions.

2.2 The detector landscape

Response/example-level F1 on RAGTruth, each figure resolved to the paper that originates it:

System	Type	Backbone	RAGTruth F1	Primary source
RAG-HAT (Song et al., 2024)	decoder, DPO-tuned	Llama-3-8B	83.9%	Song Table 2 (P 87.3 / R 80.8)
RAGTruth baseline (Niu et al., 2024)	decoder, fine-tuned	Llama-2-13B	80.7%	Niu Table 5 (P 88.6 / R 74.1)
LettuceDetect-large (Kovács & Recski, 2025)	encoder, token-level	ModernBERT-large	79.22%	Kovács & Recski Table 2
LettuceDetect-base	encoder, token-level	ModernBERT-base	76.07%	Kovács & Recski Table 2
Prompted GPT-4-turbo	prompted decoder	GPT-4-turbo	68.3%	Niu Table 5 (P 54.8 / R 90.5)
Luna (Belyi et al., 2024/2025)	encoder, sliding window	DeBERTa-v3-large	65.4%	! secondary only — Kovács & Recski Table 2
SelfCheckGPT w/ GPT-4-turbo	sampling-based	GPT-4-turbo	60.5%	Niu Table 5 (P 49.5 / R 77.7)

The ordering settles what the encoder track is: **the cheap frontier, not the frontier**. RAG-HAT’s 8B-decoder DPO pipeline and the benchmark’s own fine-tuned Llama-2-13B baseline both outscore every encoder here. Two caveats attach to the table rather than to any system in it. Luna’s figure is taken from a secondary source — its paper releases no code or weights — and is labelled so wherever it appears. RAG-HAT shares **five of its seven authors** with RAGTruth’s (Niu, Wu, Zhu, Zhong, Song), a caveat about independence, not about the figure. **All figures in this table are response-level**. This paper’s own bound (§6) is stated at the span level first, with response-level as a corroborating instance — the coarser granularity here, and the finer granularity there, should not be read as directly comparable numbers. The claim-decomposition family — AlignScore (Zha et al., 2023), MiniCheck (Tang, Laban & Durrett, 2024), and RefChecker (Hu et al., 2024) — informs this project’s own baseline design but reports no RAGTruth figures and cannot anchor a comparison here.

This table deliberately excludes this project’s own instrument, arm b (§5.1, response-F1 76.31%): it is the replication baseline for the training-side ablation, not a system submitted for comparison against the literature, and placing it in a table of externally published results would misrepresent what it is.

2.3 On the comparability of published RAGTruth figures

Assembling the comparison above required resolving each figure to its originating paper. Three discrepancies emerged. We report them because they bear on how such tables should be read, not because three cases support a general claim.

First, prompted GPT-4-turbo is reported at three different response-level F1 values on RAGTruth: **68.3** by Niu et al. (2024), the benchmark’s own authors; **76.7** by Song et al. (2024); and **63.4** by Kovács and Recski (2025) — a spread of 13.3 F1 points. The aggregation conventions differ or are undefined. Niu et al. describe their overall column as an *average* across task types; Song et al. label theirs OVERALL without definition; Kovács and Recski state no convention. We could not reconcile the three from the published text.

Second, SelfCheckGPT over gpt-3.5-turbo is reported at **36.6** by Niu et al. and **58.8** by Kovács and Recski — a spread of 22.2 F1 points, the largest we found, and one where recall diverges too (28.0 versus 71.9). Aggregation convention does not move the recall of the same predictions, so that explanation is unavailable here; we could not reconcile the two and do not claim to know the cause.

Third, the Llama-2-13B and RAG-HAT rows in Kovács and Recski’s comparison table carry precision and recall identical to Song et al.’s Table 2 (76.9 / 80.7 and 87.3 / 80.8 respectively), including the **78.7** F1 that appears in Song et al. for the RAGTruth baseline rather than the **80.7** that Niu et al. self-report for the same system. This is consistent with figures being carried across rather than recomputed — though it is not proof of it. Independent evaluation could coincide, and neither paper states which was done.

We draw no conclusion about the comparability of the RAGTruth literature as a whole; three cases cannot support one. What they suggest is narrower: that verification of external figures against originating sources may be undervalued in this literature, and that a systematic audit — resolving every published RAGTruth figure to its primary source and stated evaluation protocol — would be worth conducting. That is beyond this paper’s scope. We contribute only the observation that prompted the question.

2.4 `implicit_true` has no literature

The conceptual distinction this paper turns on — between a claim’s *faithfulness* to its source and its *factuality* in the world — is not new. Maynez, Narayan, Bohnet & McDonald (2020) established it for abstractive summarization: a generated claim can be unfaithful to its source document while still being true of the world, and conflating the two overstates or understates a summarizer’s hallucination rate depending on which direction the conflation runs. RAGTruth’s ungrounded-but-true / ungrounded-and-false split is the same distinction, applied to retrieval-augmented generation instead of summarization. What we claim as novel is narrower than the distinction itself: **the conceptual distinction is established; the benchmark ships a machine-readable field encoding it; no published RAGTruth evaluation conditions on that field.**

The field this paper audits was named and discussed in the **paper**, not only added to the data. RAGTruth’s corpus README dates the field to February 2024; the ACL paper (Niu et al., arXiv:2401.00396, §3.4 “Annotations for Adaptive Evaluation” and its “Implicit Truth” subsection) discusses the concept by name and reports per-task, per-model `implicit_true` span counts in Table 10 — checked against the full text on 2026-07-25. What the paper does not do, and what no source in our review found any system doing, is report a **detection metric** — precision, recall, or F1 — conditioned on the field. The distinction this paper turns on is discussed-and-counted versus scored-on, not mentioned versus unmentioned, and the claim

above is scoped to that: no published RAGTruth evaluation conditions its detection metric on `implicit_true`.

`due_to_null` receives the same treatment in the same paper section, and is worth reading alongside it rather than as a separate, stronger precedent than it is. §3.4’s general framing — annotations of this kind “enable users to adopt various evaluation strategies tailored to their specific application circumstances” — covers `implicit_true` and `due_to_null` under one heading, not `due_to_null` alone; we found no sentence in the paper, its appendices, or the vendored baseline code that implements an include/exclude scoring toggle for either field specifically. The `due_to_null` subsection instead states that the authors’ own subsequent detection prompts “will be aligned with this standard” — i.e., `due_to_null` spans are included as hallucinations in their own experiments, not optionally excluded. What this paper proposes for `implicit_true` — report the metric stratified, so a reader can see which subclass a number is about — is therefore not a request to extend an existing, implemented precedent from one field to the other. It is consistent with the stated intent of RAGTruth’s own “Adaptive Evaluation” framing for annotations of this kind, applied to a field that framing already names but that no published evaluation, including the benchmark’s own, operationalizes.

What is verified about the field’s handling, and what is not:

System	Field handling	Basis (checked 2026-07-25)
RAGTruth reference baseline	discards both fields	Verified in code — buckets by <code>label_type</code> alone
LettuceDetect	discards both fields	Verified in code — <code>preprocess_ragtruth.py</code> reads <code>start</code> , <code>end</code> , <code>label_type</code>
Luna	unknown	UNVERIFIED — no public code or weights
RAG-HAT	unknown	UNVERIFIED — no public code or weights

The claim is exactly the top two rows; we do not extend it to the leaderboard as a whole.

3. Methods

Three subsections, deliberately kept apart so that pre-registered and post-hoc work are visibly distinct: the annotation audit that establishes the central finding (§3.1), the pre-registered ablation behind §5’s training-time attempt (§3.2), and the exploratory re-scoring behind §6’s case study (§3.3).

3.1 Annotation audit computation

The audit operates on RAGTruth’s released `response.jsonl` and `source_info.jsonl`, one row per (response, gold span), joined to task type through `source_id`. It is a direct enumeration over 17,790 responses and 14,289 gold spans; the full computation is ~40 lines of standard library Python. It is not paraphrased here: it is reproduced verbatim as Supplement S1 (docs/research/02-implicit-true-audit.md, §5 “Exact computation,” in the repository named in the Data and Code Availability statement), so that every count below can be re-derived from the raw corpus without this project’s other code.

Three definitional choices govern the numbers.

The flag predicate. The pipeline predicate is `is_implicit_true_span()` — strict flag, defined as `implicit_true` and not `due_to_null`. It was originally named `is_noisy_span()`; the rename accompanies the reframing in §1 and §4, and the identifier is stated here because the earlier name appears in this project’s commit history. Raw `implicit_true` alone is reported alongside the strict flag throughout, because the two differ by only 6 spans.

Character mass is the right denominator, and it is not the span count. Span-level scoring on this benchmark is character-overlap micro-F1, so a long span counts for more than a short one, and any bound expressed against a span metric must be computed over character mass. `implicit_true` covers **13.5% of gold spans by count** and **14.56% of gold-span character mass** — two different quantities. An earlier version of this work quoted 13.5% as the character-mass figure; that was a conflation of the two, corrected here, and nothing downstream depends on which is quoted because the pipeline flag is computed per token from the spans directly. RAGTruth contains overlapping annotations (115 of 17,790 responses, taking span offsets as half-open `[start, end)` intervals), so summing `end - start` over raw spans double-counts overlaps: the §1/§4 figures are raw sums, while the pipeline’s per-token flag uses union-normalised regions and is the overlap-free version — 14.49% of train positive tokens against 14.53% of raw flagged character mass.

Flagging runs against raw spans, not normalised ones. A token is flagged only if *every* raw gold span covering it is flagged. A token backed by any unqualified annotation keeps full weight. The audit enters the pipeline as an auxiliary column (`is_implicit_true`, emitted alongside labels) and never as a label edit; labels, metrics, and the test set are untouched at every stage of this paper.

3.2 Pre-registered ablation protocol

ACWS — Annotation-Confidence-Weighted Supervision — down-weights flagged positions in the training loss by a factor λ , leaving labels and evaluation untouched:

$$\mathcal{L} = \frac{\sum_t (w_t \cdot \ell_t)}{\text{clamp}(\sum_t w_t, 10^{-8})}$$

ℓ_t = per-token cross-entropy (`reduction="none", ignore_index=-100`)

$$w_t = \begin{cases} 0 & \text{where label} == -100 \text{ (context / special / padding)} \\ \lambda & \text{where the token is flagged} \\ 1 & \text{elsewhere} \end{cases}$$

Two boundary properties are unit-tested rather than asserted: $\lambda = \mathbf{1.0}$ **reduces exactly to plain mean cross-entropy** (the default path does not route through the weighted branch, so it stays bit-identical to the pre-ACWS model), and $\lambda = \mathbf{0.0}$ **is exactly loss-masking**. $\lambda = 0$ was later run, at seed 42 only, as a single-point check on the null — §5.3 reports it; §8 records the single-seed caveat that check carries.

What ACWS asks the model to exploit is deliberately partial. The model has no way to verify, from context and response text alone, whether an unsupported claim is true — RAGTruth’s context is exactly what the claim is unsupported by, so there is no oracle for correctness at inference time, and none is assumed here. What down-weighting can plausibly teach is narrower: a correlate of the ungrounded-but-true subclass that survives in the model’s own parametric world-knowledge priors — the signal that makes some unsupported claims *look* more plausible than others, independent of whether the context confirms them. Whether that partial, imperfect signal is strong enough for loss down-weighting to extract is exactly what the ablation tests. A

null result does not mean the model cannot distinguish true from false in general; it means this intervention did not surface whatever prior-knowledge correlate exists, at any of the settings tested.

ACWS could not be tested alone. A code audit had separately found this project’s detector deviating from LettuceDetect’s published recipe on four axes at once — learning rate, effective batch, epochs, and checkpoint-selection metric — so testing ACWS on top of a divergent recipe would have confounded the two. Hence three arms, b and c differing in exactly one thing:

Arm	Description	lr	eff. batch	epochs	checkpoint metric	λ
a	the then- production model	2e-5	16	8	response F1	1.0
b	faithful LettuceDetect- recipe replication	1e-5	8	6	token F1	1.0
c	arm b + ACWS	1e-5	8	6	token F1	0.25

Scoring is stratified into five blocks joined back to the raw test slice. Two matter here: official, char-overlap span and response P/R/F1 scored identically to training, and **clean_span**, the same span metric with flagged intervals subtracted from *both* gold and predictions — the metric the hypothesis is actually about. The emitted keys `clean_span` and `noisy_char_mass_share` are frozen deliberately: they are the pre-registered on-disk contract of the rule below, and renaming them to match §1’s terminology would sever traceability to the rule as written before the arms ran.

The decision rule was written into the harness before any arm was run, and printed as PASS/FAIL, so the verdict was not eyeballed afterward. Adopt (c) over (b) iff all three hold:

```
clean_span_f1(c) > clean_span_f1(b)
response_f1(c) > response_f1(b)
official_span_recall(b) - official_span_recall(c) ≤ noisy_char_mass_share
```

The third clause is the honesty clause: ACWS is *expected* to cost some official span recall, because it deliberately teaches the model not to predict over flagged text the official metric still scores as gold, and the rule grants a budget for that cost.

That tolerance was computed from the data at runtime, never hardcoded. This is what makes the rule binding rather than decorative. The budget is the flagged fraction of official gold character mass, recomputed by the harness on every run, so it could not have been widened after the fact to accommodate a larger recall loss than the rule intended. A hardcoded constant would leave a reader no way to tell a pre-registered tolerance from a post-hoc one.

This runtime figure is **9.557%**, not the 9.49% quoted in §4 and §6. Both are “flagged share of gold character mass” over the same underlying spans, but over two different representations of them: §4/§6’s 9.49% is computed directly from the released corpus’s raw span offsets (85,877 characters, §6.1); the harness’s 9.557% is computed from each arm’s own `gold_spans` field in

its prediction dump — the character spans as reconstructed through this project’s tokenization pipeline, which need not exactly reproduce the raw corpus’s character count. The decision rule uses the runtime figure because it must be self-computed from what the harness has on hand at evaluation time, not sourced from a number quoted elsewhere in this paper; §4 and §6 use the raw-corpus figure because they are census claims over the released data itself. Neither figure is the “wrong” one for its purpose, and the 0.07-point gap between them changes no verdict in §5.2.

Before any arm was trusted, the harness had to reproduce the *published* numbers of the already-released model (arm a) from its prediction dump. It did — span-F1 **0.5114** against a published 0.5113, response-F1 **0.7619** against 0.7619 — and only then were b and c scored.

One thing about this protocol must be stated head-on rather than left for a reader to find. **The hypothesis it pre-registers was premised on a misreading.** ACWS was designed under a reading of `implicit_true` as marking, in the original decision record’s words, “annotator-acknowledged noise” — positives the labels had got wrong. §1 and §4 refute that reading, and it is withdrawn. The premise was wrong; the experiment was not invalidated by it, because the arms, the recipe, the decision rule, the reproduction gate, and every number they produced are independent of why the hypothesis seemed worth testing. What changes is what a null result *means* (§5). This is precisely what pre-registration is for: the record shows what was believed and when, legibly enough to be corrected in public rather than quietly re-motivated after the fact.

3.3 Post-hoc stratified re-scoring

The analysis in §6’s case study is **exploratory, not pre-registered**, and is reported as such. It was designed after the arms had run, in response to a question the audit raised but did not answer.

The cohort is the test set’s “Subtle-only” responses: those with at least one gold span where *every* gold span’s `label_type` begins with `Subtle` — no Evident spans present. There are 77, all gold-hallucinated by construction. It splits disjointly into **all-flagged** responses, where every gold span is `implicit_true` ($n = 47$), and **authentic-Subtle** responses, where at least one gold span is not ($n = 30$). Arm b’s committed per-row prediction dump is joined back to the raw span metadata by the same pattern the ablation harness uses — merged dataframe, filter to test, reset index, positional index equals the dump’s `row_index` — and the response-level miss rate (a gold-hallucinated response predicted negative) is computed over each subset. The join is self-checking: it reproduces the previously published 48.1% headline figure exactly, and the 47/77 split independently matches the audit’s count. The script prints to stdout and emits no metrics file, so this result resolves to a script plus a committed prediction dump rather than to a JSON artifact; we say so rather than let a uniform citation style imply otherwise.

The census/inference boundary is declared here, and it governs how §4 and §6 report every number. The RAGTruth test set is this paper’s object of study, not a sample drawn from a population. A claim that counts something *about that test set* — how many spans carry the flag, how much character mass they cover, what a miss rate is over a closed, fully enumerated cohort — is arithmetic, and takes **no confidence interval**, because nothing is being estimated. The moment a sentence generalizes beyond this benchmark — asserting something about how a model behaves on a *kind* of input — it changes register, and must carry its uncertainty. §6 contains one claim of each type, side by side, and labels them. Reporting a census figure with an interval would invite a reader to dismiss an exact result as noise; reporting an inferential claim without one would assert what the data cannot support. We do neither.

4. The conflated label class

This section establishes the paper’s central finding: the structured conflation that §5 and §6 each build on. Everything in it is a census over RAGTruth as released: exact counts over a closed corpus, carrying no intervals, per §3.3.

Across the whole corpus, **1,928 of 14,289 gold spans carry `implicit_true`** — 13.5% by count, and 110,171 of 756,461 characters, **14.56% of gold-span character mass**. Under the strict flag (excluding the 6 spans that also carry `due_to_null`) the figures are 1,922 and 14.53%. That is the magnitude. Its *structure* is what makes it a conflated class rather than a scattering.

The flag is a severity qualifier, and the annotators’ own intensity markers show it. RAGTruth’s free-text meta comments open with a hallucination-intensity prefix. Flagged and unflagged gold spans have close to inverted distributions:

Span set	n	LOW	HIGH	none / other
<code>implicit_true = True</code>	1,928	90.6%	0.8%	8.6%
all other gold spans	12,361	4.4%	46.4%	49.2%

The predicate is exact, not approximate, and worth stating precisely because a plausible variant changes the numbers: a span counts as LOW (respectively HIGH) iff `meta.upper().startswith("LOW")` ("HIGH"), with no leading-whitespace stripping. Stripping before the check — an equally defensible reading — moves 17 of 1,928 flagged spans (0.88%) and 70 of 12,361 unflagged spans (0.57%) whose meta string opens with a literal newline before the tag. None of the unflagged movement is into LOW: flagged-LOW moves 90.6% → **91.5%**; unflagged-LOW is unchanged at **4.4%** (545 spans under both predicates); unflagged-HIGH separately moves 46.4% → 47.0%. The flagged/unflagged LOW gap — the comparison the finding rests on — is therefore fully robust to predicate choice, and the stripped variant states it slightly more cleanly (91.5% against 4.4%) than the unstripped one does. We report the unstripped predicate as primary because it is what §5’s script and the pipeline’s own `is_implicit_true_span()` companion code actually run, not because it is the only defensible choice.

Nine in ten flagged spans are marked low-intensity by the annotator who labelled them, against fewer than one in twenty elsewhere. Read alongside the README’s definition and the comments themselves — *“these details are correct, however, not directly mentioned in the passages”* — this is what rules out reading the field as a retraction. It marks a subclass *within* the positive class, not an exception to it.

The comparison above is confounded by span kind, and a narrower cut removes the confound. RAGTruth’s four `label_type` values split into two kinds — Baseless Info (“introduction of new information”) and Conflict — and the two use different annotator vocabularies for severity. Conflict spans structurally almost never carry a LOW/HIGH prefix at all: 99.7% of Evident Conflict and 100% of Subtle Conflict spans fall in none/other, because conflict annotations are written as “EVIDENT CONFLICT: ...”, a different comment convention that carries no severity gradient. Including Conflict spans in the “all other gold spans” row therefore pads that row’s none/other share for reasons that have nothing to do with severity. Restricting the comparison to Baseless Info spans only — where both LOW and HIGH are live options either way — removes that confound: of 1,921 flagged Baseless Info spans, **90.9% are LOW**; of 6,843 unflagged Baseless Info spans, **83.7% are HIGH** and only **8.0% are LOW**. The inverted distribution survives the narrower, unconfounded cut — if anything the flagged/unflagged gap on LOW alone (90.9% against 8.0%) is a cleaner statement of the same severity split than the original all-gold-spans comparison, which mixed in a span kind that could not have scored LOW regardless of true severity.

The subclass is concentrated in one span type, not spread across four.

label_type	spans	flagged	share
Evident Baseless Info	6,237	60	0.96%
Evident Conflict	5,324	6	0.11%
Subtle Baseless Info	2,527	1,861	73.64%
Subtle Conflict	201	1	0.50%

Nearly three quarters of “Subtle Baseless Info” is ungrounded-but-true content. That category is therefore not a homogeneous detection target: it is predominantly ungrounded-but-true with a minority of ungrounded-and-false cases mixed in. Any figure reported over it — and Subtle/Evident is the benchmark’s own severity axis, carried in every gold span’s `label_type` — is a figure over a mixture whose composition is nowhere stated.

Part of this concentration is plausibly built into RAGTruth’s own taxonomy rather than discovered by us, and that costs the argument little to admit. Content that is true of the world but absent from the context is, almost by definition, not a *contradiction* of the context — so it is structurally more likely to land in “Baseless Info” than “Conflict” — and it is less likely to read as an evident, checkable error than a subtle one. Some of the 73.64% and 90.6% figures above may therefore restate the taxonomy’s geometry rather than reveal a concentration nobody expected. What survives that reading intact, and what this paper’s contribution rests on, is narrower and does not depend on the concentration being surprising: (i) the field exists, is machine-readable, and is discarded by every system whose preprocessing we could inspect (§2.4); and (ii) it covers 9.49% of test gold character mass, enough to bound official recall (§6). Nobody was arguing the flag was randomly distributed; the paper’s claim does not need it to be.

It is also concentrated by task type, which matters because RAGTruth results are routinely reported per task:

Split	Task type	flagged chars / total	share
train	QA	63,280 / 305,474	20.72%
train	Data2txt	30,008 / 262,339	11.44%
train	Summary	8,732 / 102,771	8.50%
test	QA	5,557 / 31,622	17.57%
test	Data2txt	1,855 / 36,264	5.12%
test	Summary	739 / 17,991	4.11%

On the test split, ungrounded-but-true content is more than four times as dense in QA gold mass as in Summary gold mass (17.57% against 4.11%). A per-task comparison on RAGTruth is thus not comparing like with like: the QA column is substantially more about the ungrounded-but-true subclass than the Summary column is. This uneven distribution is also the reason filtering flagged spans out of the *test* set was rejected as an option — it would change the benchmark by task type unevenly and break comparability with every published RAGTruth number. Nothing in this paper alters the test set.

The contrasting case is `due_to_null`, and it is instructive. RAGTruth’s other span-level metadata field marks spans where the model invented a value for a null field: **all 1,642 sit in Data2txt, 98.3% of them Evident Baseless Info**. Those are ungrounded-and-false hallucinations by any reading — high-severity positives that must keep full weight, which is

why this project’s flag predicate excludes them even where `implicit_true` is also set. The two fields are not interchangeable metadata. One marks the ungrounded-but-true subclass and one marks a severe one, and only the second has an authors’-provided scoring affordance (§2.4).

One split-level difference is worth stating plainly and then leaving alone. The flagged subclass is **more concentrated in train (14.49% of positive tokens) than in test (8.95%)**. We report it as a methodological fact and decline to recruit it as support for either §5 or §6, in either direction: §5’s training-time cost was measured against train’s higher concentration, and §6’s evaluation-time bound is computed over test’s lower one, so the two sections are not even describing the same density. Sensitivity to a subclass at training time is not required to scale with that subclass’s concentration, and we have no measurement that would establish it does either way. §5’s dose-response result is scoped to the settings tested, and this table does not widen it.

5. Attempting to exploit the conflation at training time: the ACWS null

The training-side question is whether a model can be taught to discount the ungrounded-but-true subclass by the simplest available mechanism — down-weighting those positions in the loss. This does not ask the model to verify truth it has no way to check; it asks whether a partial, correlational plausibility signal — grounded in the model’s own parametric priors, not in ground-truth access — can be exploited well enough by that mechanism to move a downstream metric. Under the pre-registered rule of §3.2, at $\lambda = 0.25$, it could not.

5.1 Arm b: the recipe fix, isolated

Arm b is the LettuceDetect-recipe replication with no ACWS ($\lambda = 1.0$), and it exists to give arm c a clean baseline. It is also, on its own, the larger effect in this ablation. Against arm a — the same architecture and data under this project’s earlier recipe — arm b reaches **span-F1 0.5321** against 0.5113, a gain of 2.1 points, with response-F1 **0.7631**, response precision 0.8359 against 0.7873, and a false-positive rate on faithful responses of 7.4% against 10.7%. ADR-020 attributes that gain to checkpoint selection on span-level rather than response-level F1, and §7.3 gives the structural reason it is the plausible cause. The attribution is an interpretation, not an isolated measurement: arm b changed four recipe axes at once — learning rate, effective batch, epochs, and selection metric (§3.2) — so the ablation does not partition the 2.1 points among them. Nothing in that result involves `implicit_true`, and we do not present it as part of the paper’s argument — it is the control arm, reported so that the comparison arm c is judged against is visible.

5.2 Arm c: ACWS at $\lambda = 0.25$

All three of arm c’s seeds now resolve to committed artifacts. Seed 42 and seed 123 were recovered via Hub inference after the original run’s metrics-save step was lost — training was genuine, only the save step failed. Seed 456 is a fresh, full run (RunPod, eager attention) with a complete metrics file and prediction dump. No arm in this ablation rests solely on a decision record any longer.

Arm c was **designed** to differ from arm b in exactly one parameter, `--implicit_true_weight 1.0 → 0.25` (§3.2’s table). The committed metrics files do not fully bear this out, and the discrepancy is disclosed here rather than left for a reader to find:

Run	epochs	micro-batch $\times$ accum (eff. batch)	attention	λ
arm b, seed 42	6	4 $\times$ 2 (8)	sdpa	1.0
base, seed 123	4	4 $\times$ 2 (8)	sdpa	1.0
base, seed 456	4	4 $\times$ 2 (8)	sdpa	1.0
arm c, seed 42	unrecorded	unrecorded	unrecorded	0.25
arm c, seed 123	unrecorded	unrecorded	unrecorded	0.25
arm c, seed 456	6	1 $\times$ 8 (8)	eager	0.25
arm d, seed 42	6	1 $\times$ 8 (8)	eager	0.0

Effective batch size is uniform (8) across every run that records it. Epochs and attention implementation are not: arm c seed 456 ran 6 epochs on eager against arm b seed 42’s 6 on sdpa, but against the base seeds it is compared to in the 3-seed mean (123, 456 at 4 epochs each) it ran 6. Arm c seeds 42 and 123 record no hyperparameters at all beyond the seed value — the same configuration-unrecorded signature Appendix A discloses for large-backbone seed 123, applied here to the two seeds that, together with seed 456, carry this section’s headline verdict.

One direction is worth stating plainly. Where recipe details are recorded, arm c seed 456 had **more** epochs available than its matched base-arm anchors (6 against 4) and still lost to arm b on both clauses at that seed (§5.2 below). If recipe drift favored either arm here, it favored arm c — and arm c still failed the rule. The rejection is more robust for it, not less.

Evaluated on the pre-registered rule, using each arm’s mean across its three seeds (42, 123, 456, paired by seed value though not confirmed recipe-matched on every seed):

Clause	Arm b (mean, n=3)	Arm c (mean, n=3)	Δ (c – b)	Verdict
clean_span_f1(c) > clean_span_f1(b)	0.5308	0.5197	–0.0111	FAIL
response_f1(c) > response_f1(b)	0.7637	0.7587	–0.0050	FAIL
official span-recall drop $\leq$ noisy char-mass share	—	—	—	not reached

Per seed:

Seed	clean_span F1 (b)	clean_span F1 (c)	Δ	response F1 (b)	response F1 (c)	Δ
42	0.5307	0.5171	–0.0137	0.7631	0.7623	–0.0008
123	0.5281	0.5209	–0.0072	0.7637	0.7590	–0.0047
456	0.5336	0.5212	–0.0125	0.7643	0.7549	–0.0094
mean	0.5308	0.5197	–0.0111	0.7637	0.7587	–0.0050
range	0.0055	0.0041	—	0.0012	0.0074	—

Source: *results/seed_aggregate_arm_c_vs_arm_b.json* (response F1,
via *scripts/aggregate_seeds.py --base results/arm_b_metrics.json*

`results/base_seed123_metrics.json results/base_seed456_metrics.json --large results/arm_c_seed42_metrics.json results/arm_c_seed123_metrics.json results/arm_c_seed456_metrics.json) and results/clean_span_seed_variance_full.json (clean-span F1, via scripts/ablation_report.py --arm b42=... --arm b123=... --arm b456=... --arm c42=... --arm c123=... --arm c456=...).`

Both clauses fail, on every seed: arm c beats arm b on neither clean-span F1 nor response F1 at seed 42, 123, or 456 — 0/3 and 0/3. The rule rejects arm c unambiguously.

This supersedes an earlier, weaker reading of the same decision. Before seed 123 and seed 456 had real artifacts, arm c’s numbers resolved only to ADR-020’s single-seed record, which reported response F1 0.7633 against arm b’s 0.7631 — a nominal pass on clause 2, by +0.0002. The recovered seed-42 artifact does not reproduce that figure: its response F1 is 0.7623, which fails clause 2 at that seed (−0.0008 against arm b), and its clean-span F1 (0.5171) is also below ADR-020’s originally reported 0.5262 for the same nominal setting — a 0.0091 gap on the metric the hypothesis is actually about. Response F1 is close between the two records (0.7633 vs 0.7623); clean-span F1 is not. We do not know the cause of the discrepancy; it is disclosed as an unresolved, non-load-bearing footnote in §8 — non-load-bearing because it changes no overall decision. Clause 1 fails under both readings, so arm c is rejected either way, and at the 3-seed level clause 2 fails on the mean and on every individual seed regardless of which seed-42 reading is used, since even ADR-020’s more favorable seed-42 figure is dwarfed by seeds 123 and 456’s larger, unambiguous losses.

5.3 Reading the null

Two things changed since this section first went to draft: arm c’s three seeds are now all real, matched one-for-one to arm b’s three seeds (§5.2), and a fourth arm — d, $\lambda = 0.0$, seed 42 — has been run. Between them they close one of the constraints this section used to list as open and sharpen a second into something more specific than the flat null originally reported here.

The λ sweep is now three points, not one. Read across $\lambda = 1.0$ (arm b) $\rightarrow \lambda = 0.25$ (arm c) $\rightarrow \lambda = 0.0$ (arm d), at seed 42 (arm d has only that seed):

λ	Arm	clean-span F1	response F1
1.0	b (seed 42)	0.5307	0.7631
0.25	c (seed 42)	0.5171	0.7623
0.0	d (seed 42)	0.5113	0.7487

This sweep is not λ -isolated, and that is disclosed here rather than left implicit.

As §5.2’s configuration table shows, arm c seed 42’s hyperparameters beyond the seed value are unrecorded, so the sweep’s middle point cannot be confirmed to match either endpoint on epochs or attention implementation. The two endpoints do differ on both axes: arm b seed 42 ran 6 epochs on sdpa; arm d seed 42 ran 6 epochs on eager, with a different micro-batch/accumulation split (1×8 against 4×2 , same effective batch of 8). We do not believe attention-implementation or micro-batch geometry differences of this kind plausibly account for a move of 0.019 F1 — these are numerics-level implementation choices, not recipe changes expected to shift accuracy at that scale — but that is reasoning about plausibility, not a claim verified with a control run holding λ fixed and attention implementation varied, and no such run is planned (§8).

With that caveat stated, both metrics decline monotonically as λ drops — the direction never reverses across the three points. The total move from $\lambda = 1.0$ to $\lambda = 0.0$ is **−0.0194**

clean-span F1 and **−0.0144 response F1**. For scale: arm b’s own three-seed range is 0.0055 clean-span F1 and 0.0012 response F1 (§5.2) — though that range is itself not a pure seed-noise estimate, since those three runs differ in epoch cap (6/4/4; the same asymmetry disclosed in Appendix A for the parallel large-backbone comparison). We report the raw ranges and the raw deltas side by side rather than compute a noise-floor multiple from an $n = 3$ range, which would overstate what three points can establish about dispersion. The full-range decline is several times larger than arm b’s own seed-to-seed spread on both metrics; that is not the signature of a flat, insensitive model. *Source: results/clean_span_seed_variance_full.json, arm d42, via scripts/ablation_report.py --arm b42=... --arm c42=... --arm d42=results/arm_d_lambda0_seed42_preds.json.*

This resolves the confound the earlier draft of this section left open — in neither of the two directions it originally offered. That draft could not distinguish “the model is insensitive to this distinction” from “the perturbation falls below the resolution of the training process,” and named $\lambda = 0$ as the run that would separate them. It has now been run, and it rules out both:

- **Not model insensitivity.** An insensitive model would show flat clean-span and response F1 across λ . Instead both decline monotonically, by margins several times the seed-noise floor established in §5.2. The model is demonstrably sensitive to this intervention.
- **Not an intervention too small to register.** $\lambda = 0$ is not a larger nudge in the same direction as $\lambda = 0.25$ — it is the maximum possible strength of this intervention by construction: exact loss-masking, where the flagged tokens contribute nothing to the gradient. If the null at $\lambda = 0.25$ reflected an effect too faint to detect, the strongest possible version of the same intervention was the one run capable of surfacing it. It did not surface a benefit. It surfaced a larger cost.

What the evidence instead supports, stated as interpretation and not as a new pre-registered result: the model was extracting productive gradient signal from the flagged, ungrounded-but-true tokens during training, and down-weighting or masking them removes real signal rather than removing noise. This is the opposite of ACWS’s premise — that these positions were safe to discount because they correlated with a subclass the model did not need to learn from — and a monotonic, three-point decline is what a signal-removal account predicts and a pure-noise account does not.

Two caveats apply here: one to the sweep’s shape, one to its endpoints (disclosed above). Arm d is a single seed. The $\lambda = 0.25 \rightarrow 0.0$ step alone (clean-span F1 0.5171 $\rightarrow$ 0.5113, −0.0058) is the same order of magnitude as arm b’s own three-seed range (0.0055) — on this evidence alone, that specific half-step is not individually distinguishable from seed noise. What exceeds arm b’s own range by a wide margin is the full $\lambda = 1.0 \rightarrow 0.0$ move (−0.0194 against arm b’s seed-42 value specifically, −0.0195 against arm b’s three-seed mean). The claim rests on the full range and its monotonic shape across three points, not on the middle segment considered in isolation — and it is read alongside the attention-implementation/micro-batch caveat on the endpoints stated above, not in place of it.

Four constraints remain: one resolved outright, one resolved into a different and more specific finding than either of its original candidates, and two unchanged.

Resolved: one seed $\rightarrow$ three seeds, matched. Arm c’s clean-span F1 and response F1 both lose to arm b’s on every one of three matched seeds (§5.2) — no longer a single-seed result subject to that seed’s own draw.

Resolved, differently than either original candidate predicted: the insensitivity/too-small confound. As above — neither “the model can’t tell” nor “we didn’t push hard enough” survives the $\lambda = 0$ run. The null has a specific, monotonic shape that neither original account

predicts. A signal-removal account — the model was using the flagged tokens’ gradient productively, and down-weighting or masking them removes real signal — predicts exactly this shape. So does a second account this data cannot rule out: **supervision inconsistency**, in which telling the model that surface-identical unsupported claims sometimes count toward the loss and sometimes don’t (a distinction it cannot observe at inference, since RAGTruth’s context is exactly what the claim is unsupported by) degrades performance in proportion to how strongly that inconsistency is enforced, independent of whether the down-weighted tokens carried extractable signal. Both accounts predict a monotonic decline; this data does not distinguish between them, and no experiment reported here does either.

Unchanged: the λ grid is still coarse, and one architecture is tested throughout. Three points (0.0, 0.25, 1.0) establish direction and rough magnitude but not the sweep’s shape between them — whether the decline between $\lambda = 0.25$ and $\lambda = 1.0$ is linear, front-loaded, or back-loaded is not observed, and no intermediate λ was run. ModernBERT-base is the only backbone tested.

Unchanged: checkpoint selection is on the wrong metric for what the hypothesis asks. Arms b, c, and d all select checkpoints on token-level F1 (§3.2), which scores flagged tokens the same as any other positive; the ACWS hypothesis is specifically about clean-span F1, which does not. This confound is structural — no seed count or λ value resolves it — and stays acknowledged, not fixed.

Unchanged: nothing here licenses a general robustness or mislabelling claim. This is not evidence about a model’s tolerance for mislabelled training data, and cannot be — §4 establishes that the flagged subclass is not mislabelled. What was tested is whether a real, low-severity error subclass can be discounted by loss reweighting, at any strength from full weight to none. At every strength tested, it could not — and the strongest version of the attempt made things worse.

The null is better supported now than the version of this section originally reported, and sharper. The original claim was: at $\lambda = 0.25$, one seed, one architecture, down-weighting the ungrounded-but-true subclass produced no measurable improvement in clean-span F1 — a result a reviewer could reasonably read as underpowered, resting on a single seed and an unresolved confound the paper admitted it could not close. The claim now is: **down-weighting the ungrounded-but-true subclass, at every strength tested from $\lambda = 0.25$ to $\lambda = 0.0$ and across three matched seeds at $\lambda = 0.25$, never once improved clean-span F1 or response F1 over the unweighted baseline, and the decline is monotonic in the down-weighting’s strength.** That is not a weaker null falling short of significance; it is a consistent, dose-dependent cost. The chapter’s contribution remains the protocol around the result as much as the result itself — a decision rule fixed in code before any arm ran, a harness validated against published numbers first — but the result it now protects is no longer thin. That, and not a single -0.0045 , is what §7 carries forward.

6. Quantifying the conflation’s cost at evaluation time

§4 established that the conflation is real and structured. This section quantifies what it costs under RAGTruth’s official scoring — a direct, model-independent consequence of that structure, not a second, independent result. The reason is arithmetic rather than empirical: RAGTruth’s official scoring counts the ungrounded-but-true subclass as gold, so a detector that declines to flag it is scored as having missed it.

6.1 The span-level bound

Span-level P/R/F1 under character overlap is the metric token-level detectors on this benchmark are principally scored on, so it is where the bound bites hardest and where we state it first. Flagged spans cover 8,151 of the 85,877 characters of gold span mass in the test split — 9.49%. Therefore:

Under RAGTruth’s official scoring, any prediction set that excludes the annotator-flagged low-intensity subclass from its positives is capped at 90.5% char-overlap span recall against gold — a property of the scoring protocol, computable from the released labels alone, independent of whether any detector can or should make that exclusion.

The figure is $(85,877 - 8,151) / 85,877 = 90.5\%$. It is a census over a closed, fully enumerated test split, computed directly from the released corpus, and it carries no confidence interval because nothing is being estimated (§3.3). It is not an empirical finding about any particular system; it is a property of the benchmark that holds before any model is trained.

6.2 The response-level bound

The same argument applies at coarser granularity, and we report it as a corroborating instance rather than as a second result. A response counts as hallucinated if it carries any gold span; 49 of the 943 hallucinated test responses consist *entirely* of flagged spans. Therefore, under the same protocol-level argument, **any prediction set excluding the low-intensity subclass is capped at $(943 - 49) / 943 = 94.8\%$ response-level recall against gold.** The response-level bound is the weaker of the two — a response survives if even one of its spans is ungrounded-and-false — which is precisely why the span-level figure leads.

6.3 What the bound is and is not

The statement above is a bound on **achieving two goals at once**. It is not a ceiling on recall in isolation, and any paraphrase that drops the conditional is false: a system that flags every token still reaches 100% recall on this benchmark, trivially and uselessly. What cannot be done is to reach past 90.5% span recall *while* declining to flag ungrounded-but-true content. Every statement of the bound in this paper carries that conjunction, and we have written it out in full each time rather than abbreviate it after first use.

Two consequences follow. First, the bound is a property of the scoring function’s relationship to the gold labels, not of any system that might be scored by it. Stated in terms of a hypothetical detector’s *objective*, it is indifferent to model class, size, or training data, and a frontier-scale system is bound by it exactly as tightly as a 150M encoder.

Second, and this is the sentence in this paper with the widest reach: recall figures on this benchmark are not comparable across detectors with different implicit objectives, and that applies to every system in §2.2’s landscape table, not only the instrument used here. RAGTruth’s official scoring does not ask what a system was built to catch; it asks only whether a flagged span overlaps a gold span, of whatever severity. A system tuned to surface ungrounded-and-false errors and a system tuned to maximize official recall are scored on the same axis while optimizing different things, and nothing in a published F1 distinguishes them — the metric cannot tell a low-severity span missed by design from one missed by failure. This is not a claim that any figure in §2.2 is wrong; each is a correct report of what its official metric measures. It is a claim about what that metric cannot tell a reader: a higher published F1 does not imply a system catches more ungrounded-and-false content specifically, because the aggregate score is silent on which subclass drove the difference. Two systems separated by several F1 points could

differ almost entirely in how much ungrounded-but-true content each flags, with their handling of the ungrounded-and-false subclass nearly identical — or the reverse — and no published number distinguishes the two cases. Resolving this for any specific pair of published systems would require re-scoring their released predictions against a stratified metric, which this paper does not attempt for the literature as a whole; that is a distinct exercise from §2.3’s narrower concern about resolving figures to primary sources. We state this consequence as this paper’s clearest implication for how the field should read its own leaderboard, not as an audit of that leaderboard.

6.4 Subordinate case study: the Subtle cohort

The bound is aggregate. Where the conflation actually lands in a specific detector’s error profile is a separate, narrower question, and we answer it for one model — arm b — as a worked example. This subsection is scoped to that model and is not a second headline.

§4 showed the flagged subclass concentrates in Subtle Baseless Info. Arm b’s weakest reported cohort is Subtle-only responses, at a 48.1% miss rate. The natural inference is that part of that figure is not a real capability gap but the model declining to flag content a detector limited to ungrounded-and-false content would arguably be right to leave alone. We tested it, and it does not hold.

Cohort	n	missed	miss rate
Full Subtle-only cohort (as reported)	77	37	48.1%
Authentic-Subtle only (all-flagged excluded)	30	16	53.3%
All-flagged only (reference)	47	21	44.7%

Excluding the flagged responses raises the miss rate, from 48.1% to 53.3%. This is a census claim — exact counts over a closed 77-response cohort — and it carries no interval. The direction is the point: the flagged subclass was *diluting* the headline figure toward a better-looking number, not inflating it. Arm b’s real difficulty with authentic Subtle hallucinations is understated by the metric as reported, not overstated.

The mechanism that would explain this lives in a different register and is **not established**. Arm b catches 26 of 47 flagged Subtle responses (55.3%) against 14 of 30 authentic ones (46.7%) — an 8.7-point gap, on which Fisher’s exact test, two-sided, gives $\mathbf{p} = \mathbf{0.491}$. That is not distinguishable from chance. We report it because omitting it would leave the census result unmotivated, and we label it as a candidate mechanism for future work rather than a finding. The census claim is the result; the mechanism is a guess at why, and keeping them in separate registers is what stops the guess’s weak evidence from dragging down the exact result.

Two readings this data does not support, stated explicitly because both are the natural thing to conclude and both are wrong. It does **not** show that the Subtle weakness is a labeling artifact inflating the miss rate — the measured direction is the opposite. And it does **not** establish that the model is better at ungrounded-but-true cases than at authentic ones; that is true of this cohort as counted, but the p-value above is exactly the reason it cannot be asserted as a property of the model.

The case study’s reach is genuinely limited, and the limitation is worth naming rather than leaving for a reviewer. No published system reports a Subtle-stratified miss rate, so the distortion demonstrated here lands on a metric only this project reports. That is why the case study is subordinate: it illustrates where the conflation concentrates inside one detector’s errors, while §6.1’s bound is the claim with reach beyond this paper, because it constrains every system scored on this benchmark regardless of what any of them chooses to report.

7. Discussion

7.1 The training-time cost and the evaluation-time bound are not in tension

A reader could reasonably wonder whether §5’s result and §6’s bound are in tension: if the flagged subclass is large enough to cap recall at 90.5% (§6), and down-weighting it produced a real, monotonic cost at training time rather than nothing (§5.3), how can the same subclass be both a hard ceiling regardless of any model and a quantity a model is sensitive to? They are not in tension, but the resolution is not the one an earlier draft of this section offered. It is not that the down-weighted quantity was “too small to steer a training process” — §5.3 rules that reading out directly: the model demonstrably is steered by it, and removing it costs performance. The actual asymmetry is a mismatch in the size of the quantity each stage operates over, and in how much dilution each stage applies to it — not a mismatch in whether the quantity matters.

At training time, the flagged subclass enters through the loss, competing with every other supervised token for the same gradient. It is roughly 0.7–0.8% of supervised tokens, contributing that same ~0.8% share of total loss mass at $\lambda = 1.0$; down-weighting it to $\lambda = 0.25$ shifts that contribution to roughly 0.2% of total loss mass, and $\lambda = 0$ removes it entirely. §5.3 shows that removing even this small, heavily diluted a share is enough to cost the model something real: clean-span and response F1 both decline monotonically as λ drops, and the decline is largest exactly where the down-weighted share is smallest. The quantity is not too small to matter — it demonstrably does, and the model was extracting productive gradient signal from it.

At evaluation time, the same subclass enters through the scoring function, and the difference is in kind, not only in degree. It is 9.49% of test gold character mass, counted directly and fully by RAGTruth’s official metric, with no dilution across other tokens or other loss terms — over 40 times larger than the ~0.2% loss-mass share the down-weighted quantity fell to at $\lambda = 0.25$, and more than an order of magnitude larger even than the ~0.8% share it holds undiminished at $\lambda = 1.0$. Every flagged character is counted once, directly, against any detector that declines to flag it; nothing else in the scoring function competes with it for attention the way every other supervised token competes with it for gradient during training.

Put plainly: the same subclass is large enough, and undiluted enough, to cap what official scoring can report (§6) — and, separately, small enough in absolute terms, and diluted enough across a full training objective, that manipulating its loss weight measurably moves clean-span and response F1 in the wrong direction rather than not moving them at all (§5.3). Both are now real, measured effects of the same subclass; what differs is the size of the quantity each mechanism operates over and whether anything dilutes it on the way in. §6’s bound needs no model — it is arithmetic over the released corpus and holds before any system is trained, so it cannot be undermined by a training-time result in either direction. Symmetrically, §5’s finding that the model uses this subclass’s gradient productively says nothing about the scoring protocol’s arithmetic, which is fixed by the gold labels regardless of what any model learns from them.

7.2 The arm-b complication

One result cuts against the tidy version of this story, and we include it deliberately. Adopting arm b’s corrected recipe — the change §5.1 reports as a 2.1-point span-F1 gain — made the detector *worse* on exactly the cases this paper cares most about. Subtle hallucination miss rate rose from 40.3% to 48.1%, Evident from 27.0% to 30.6%, and the FP:FN ratio shifted from 0.76 to 0.46, meaning the model bought its precision by under-flagging rather than by flagging more accurately.

This is awkward, and it strengthens the argument rather than undercutting it. If Subtle per-

formance moved by nearly eight points in response to a change that has nothing to do with the annotation — a checkpoint-selection metric, a learning rate, a batch size — then Subtle detection is fragile to training decisions that are not about the phenomenon at all. Label- class conflation does not explain arm b’s Subtle weakness, and this paper does not claim it does. Subtle detection is a genuinely unresolved problem, and §6.4’s finding is that the metric reporting it is *also* distorted, not that the distortion is the whole of the difficulty. A reader who came away thinking the conflation accounts for the Subtle gap would have read us wrongly, and this complication is the clearest evidence against that reading.

7.3 A transferable lesson about checkpoint selection

The mechanism behind arm b’s gain generalizes past this benchmark, and we record it here as a Discussion observation rather than as a contribution of the paper.

The earlier recipe selected checkpoints on response-level F1 — the deployment-relevant metric, and the defensible-looking choice. Changing that selection, together with three other recipe axes, recovered 2.1 span-F1 points (§5.1), and selection is the axis with a structural reason to have caused it: response-level F1 asks only whether *any* token in a response was flagged, so it cannot distinguish a tight, well-placed span from a sloppy one that happens to overlap the gold region. Two checkpoints that are identical at response level can differ substantially at span level, and selecting on the coarse metric picks between them arbitrarily.

The general form: **when a system has both a coarse and a fine success criterion, selecting on the coarse one does not merely fail to optimize the fine one — it discards the information needed to choose between candidates that tie at the coarse level.** Any pipeline with nested granularities of correctness can fall into this, and the failure is invisible in the metric being selected on. We flag it because nothing in the coarse metric’s behaviour signalled the loss — the suppressed checkpoints scored normally on the metric being used to choose them — and because the cost was recoverable once the fine metric was consulted at selection time.

7.4 Implication for benchmark reporting

The narrow implication of §6 is that a RAGTruth recall figure does not, on its own, tell a reader what kind of error the detector is catching. A detector that reports 85% span recall may have caught predominantly ungrounded-and-false content, or predominantly ungrounded-but-true content, and the number is identical either way.

Stratified reporting would resolve this: publish the metric separately over ungrounded-and-false and ungrounded-but-true gold, alongside the aggregate. The benchmark already ships the field this requires, and its own “Adaptive Evaluation” framing (§2.4) already names annotations of this kind as intended to support exactly this — even though no published evaluation, including the benchmark’s own, has operationalized that intent for either `implicit_true` or `due_to_null`. We offer this as an implication of what we measured rather than as a recommendation we have standing to make — one audit and one null ablation do not establish what a benchmark’s reporting conventions should be, and we have not evaluated what stratified reporting would cost in practice or whether it would change any published ranking. What we can say is narrower and, we think, sufficient: the information needed to report that way already exists in the data, unused.

The following table makes the proposal concrete with a number already on hand rather than a hypothetical one. Both columns are arm b (§5.1), computed over the same test-split predictions; the second column subtracts the flagged (ungrounded-but-true) intervals from both gold and predictions before rescore, exactly as §5.2 uses to evaluate the ACWS hypothesis.

Metric	Aggregate (official)	Flagged-excluded (clean_span)
Span-F1	0.5321	0.5307

Source: aggregate column from results/arm_b_metrics.json (test.span_char_level.f1); flagged-excluded column from the clean_span block of scripts/ablation_report.py, reported in §5.2 and recorded in docs/EXPERIMENT_LEDGER.md (E6) and ADR-020. No new computation was run to produce either figure.

The two columns differ by 0.0014 — small at this scale, and in this instance the direction happens to run the other way from the concern that opened §6 (excluding flagged spans here costs recall on a subclass more entwined with the true positives than it inflates the headline). The magnitude of the gap is not the point and we do not read it as evidence either way about arm b specifically. The point is narrower: the two columns are both already computable from released annotations, neither requires retraining or additional inference, and reporting only the first — as this benchmark’s convention currently does — discards information that was sitting in the same prediction dump used to produce it.

We do not have a response-level counterpart to sit beside this table. The aggregate response-F1 (0.7631, same source) has a published analogue, but ablation_report.py’s clean_span block recomputes only the char-span metric with flagged intervals removed — it was never extended to derive a parallel flagged-excluded response-level figure, and no other artifact in this project contains one. Producing it would mean computing a new number rather than surfacing an existing one, so we leave the response-level half of this demonstration for future work and note the gap rather than approximate it.

8. Limitations

The training-side null (§5).

- **The λ grid is coarse, and only one architecture is tested.** Three points ($\lambda = 1.0, 0.25, 0.0$), all on ModernBERT-base. The shape of the decline between these anchors is not observed, and no intermediate λ was run.
- **The λ -sweep’s two endpoints are not λ -isolated.** Arm b (seed 42, sdpa, 4×2 micro-batch) and arm d (seed 42, eager, 1×8 micro-batch, same effective batch of 8) differ in attention implementation and micro-batch geometry as well as λ ; arm c seed 42, the sweep’s middle point, has no recorded hyperparameters beyond its seed value and cannot be confirmed to match either endpoint. §5.3 gives the reasoning for why a numerics-level difference of this kind is not expected to explain a move of this magnitude, but that reasoning is not verified with a control run, and none is planned.
- **Arm b’s own three-seed range is not a pure seed-noise estimate.** Seeds 42, 123, and 456 ran at 6, 4, and 4 epochs respectively — the same epoch-cap asymmetry disclosed below for the large-backbone comparison. The range used throughout §5.2–§5.3 as a scale reference reflects this asymmetry as well as seed variation, which is why it is reported as a raw range rather than as a multiplier.
- **Arm d ($\lambda = 0$) is a single seed.** The full-range decline ($\lambda = 1.0 \rightarrow 0.0$) exceeds arm b’s own three-seed range by a wide margin, but the middle segment alone ($\lambda = 0.25 \rightarrow 0.0$) does not exceed that range on the single-seed evidence available, and arm d’s own seed-to-seed variance is unmeasured. §5.3 states this caveat in full.
- **An unresolved discrepancy between ADR-020’s original arm-c seed-42 record and the recovered artifact.** ADR-020 originally reported response F1 0.7633 and clean-span F1 0.5262 for arm c at seed 42; the artifact recovered via Hub inference gives 0.7623

and 0.5171 for the same nominal run. Response F1 is close (a 0.0010 gap); clean-span F1 is not (a 0.0091 gap, larger than arm b’s own three-seed spread of 0.0055). The cause is unknown and was not investigated. Disclosed here as non-load-bearing: both readings place arm c below arm b on clean-span F1, and §5.2’s 3-seed mean rejects arm c on both decision-rule clauses regardless of which seed-42 reading is used.

- **Checkpoint selection is on the wrong metric for what the hypothesis asks** (§5.3). Arms b, c, and d all select checkpoints on token-level F1, not clean-span F1. This is structural and unaffected by seed count or λ .

The arm-b comparison (§5.1, §7.3).

- **Four recipe axes changed at once** — learning rate, effective batch, epochs, and selection metric. The 2.1-point span-F1 gain is not partitioned among them. Attributing it to checkpoint selection rests on a structural argument, not on an isolating arm.

The evaluation-side case study (§6.4).

- **n = 30 in the authentic-Subtle cohort.** A small denominator limits what one cohort can illustrate. It does not make the count inexact: no confidence interval is reported, deliberately, and an earlier $\pm 18\text{pp}$ interval was withdrawn as a misapplication of inferential statistics to a census. That withdrawal is not to be reversed.
- **The mechanism is unestablished** (Fisher’s exact $p = 0.491$) and is reported only as a candidate explanation.
- **Reach.** No published system reports a Subtle-stratified miss rate, so this case study lands on a metric only this project reports.

Related work (§2).

- **Luna and RAG-HAT are unverified.** Neither publishes code or weights, so their handling of `implicit_true` could not be checked. The novelty claim is scoped to the two systems whose preprocessing is publicly inspectable.

The scaling replication (Appendix A) and the variance anchor.

The base arm of this comparison is three fully matched, artifact-backed seeds (the same three arm-b runs used throughout §5); the caveats below are specific to the large-backbone side.

- **E10’s epoch cap is not uniform.** Seed 42 ran at 6 epochs, seeds 123 and 456 at 4; seed 123 selected its final epoch and may not have converged. The sweep is “seed, with a differing epoch cap on one arm”, not “seed only”.
- **Large seed 123’s configuration is unrecorded** and its predictions were never dumped, so stratified re-analysis covers two of three large seeds.
- **The seed-42 large checkpoint is not published to the Hub**, so the reporting-model claim is not yet externally verifiable.

Self-disclosed correction.

- **Earlier versions of this work quoted 13.5% as a character-mass share.** It is a span *count* share; the character-mass figure is 14.56%. Corrected in §3.1.

9. Conclusion

RAGTruth’s positive class aggregates two materially different error types, and the benchmark ships the field that separates them. §4 showed this is not a scattering of annotator disagreement: the conflation concentrates in one span type — 73.64% of spans typed “Subtle Baseless Info” are themselves the low-intensity, ungrounded-but-true subclass — and is unevenly distributed

by task, the structure of a real, annotator-recognized subclass. That structured finding is what the rest of the paper builds on.

Its consequence at evaluation time is exact and model-independent: flagged spans are 8,151 of the 85,877 characters of test gold span mass, so **under RAGTruth’s official scoring, any prediction set that omits the low-intensity subclass is capped at 90.5% char-overlap span recall against gold** — a property of the scoring protocol, not a claim about what any detector does, and a direct arithmetic consequence of §4 rather than an independent second result. At the coarser response level, 49 of 943 hallucinated test responses are entirely flagged, giving the corroborating 94.8% figure. Both are census counts over a closed, fully enumerated test split, computed from the released corpus and carrying no confidence interval, because nothing is being estimated. Neither depends on any model, and both hold before a detector is trained.

We also tested whether the same conflation could be exploited at training time. Down-weighting the ungrounded-but-true subclass in the loss, under a decision rule fixed in code before the arms ran, never improved clean-span F1 or response F1 over the unweighted baseline — not at $\lambda = 0.25$ across three seeds (paired by seed value with arm b, though not confirmed recipe-matched on every seed; §5.2 discloses the discrepancy), and not at $\lambda = 0.0$, the maximal version of the intervention, which instead produced a larger, monotonic decline. That dose-response rules out both candidate readings of an earlier, thinner version of this result: the model is not insensitive to the intervention, and the intervention was not too small to register. This remains a negative result, narrower than the finding it sits beside, and we claim it as such rather than as a second leg — but it is a specific, dose-dependent negative result now, not an underpowered one.

The evaluation-time bound and the training-time cost are not in tension, for the reason §7.1 gives: the asymmetry is not that the flagged subclass was too small to matter at training time — the paragraph above just ruled that reading out — but that it is a small quantity, diluted across a full training objective (0.2–0.8% of loss mass depending on λ), relative to the far larger quantity the evaluation-time bound is computed over and applies no dilution to at all (9.49% of test gold character mass; 8.95% of test positive tokens is the related but distinct train/test density figure from §4). A real training-time cost and a real evaluation-time bound coexist without contradiction, because they are two different questions asked of the same structured fact — how much removing this signal costs a model that must compete it against every other gradient, and how much this signal counts toward a metric that dilutes nothing — not one result measured twice.

We close with an implication we have not demonstrated and offer as speculation rather than as a lesson. `implicit_true` exists because RAGTruth’s annotators recorded not only *that* a span was ungrounded but *how severe* that was — and having recorded it, the field went unused by the systems trained on the data and unreported by the papers that scored on it. Annotation-confidence and annotation-severity metadata may be a generally under-used signal in benchmark construction: cheap to collect while annotators are already reading the text, and expensive to reconstruct afterward. Whether that generalizes past this one corpus is not something one audit and one null ablation can establish, and we do not claim it.

Data and Code Availability

All code, configuration, and result artifacts for this project are in a public repository: <https://github.com/hugoomez/rag-hallucination-detector>.

Every quantitative result reported in this paper resolves to a committed artifact in that repository — a metrics file and, for every arm used in a per-seed or matched comparison, a prediction

dump. Arm c’s three seeds (42, 123, 456) and arm d’s seed 42 all resolve to committed metrics files and prediction dumps under `results/`; arm c’s seeds 42 and 123 were recovered via Hub inference after the original run’s metrics-save step was lost, and are disclosed as such wherever they are used (§5.2), but they are independently re-derivable from the released checkpoints, not sourced to a decision record. ADR-020 remains the timestamped record of arm c’s original single-seed evaluation and is cited in §5.2 as the earlier reading superseded by the recovered and newly run artifacts; a small, unexplained discrepancy between that original record and the recovered seed-42 artifact is disclosed in §8 and does not affect any claim in this paper (§5.2).

Arm a’s metrics file, arm b’s metrics and prediction dumps for all three seeds, arm c’s metrics and prediction dumps for all three seeds, arm d’s metrics and prediction dump, the E10 three-seed variance anchor, and Appendix A’s scaling comparison for large seeds 42 and 456 are all committed under `results/`. Arm a has no committed prediction dump — the harness used one to reproduce arm a’s published numbers at run time (§3.2), but it was not retained — so arm a’s per-row predictions are not independently re-derivable from this repository. Appendix A’s own two partial gaps — large seed 123’s predictions were not dumped and its checkpoint was not published to the Hub — are disclosed in §8 and do not affect any claim in the paper’s main argument (§4–§7), which Appendix A is explicitly severable from.

The annotation audit underlying §3.1 and §4 is a direct, standard-library computation over the released RAGTruth corpus, reproduced in full as Supplement S1 (`docs/research/02-implicit-true-audit.md`, §5) rather than paraphrased, so every count in this paper can be re-derived from the raw corpus independently of this project’s other code.

§6.4’s Subtle-cohort miss rates resolve to `scripts/subtle_only_miss_rate.py`, run against the committed `results/token_preds_arm_b.json` prediction dump; the script emits no metrics file, so it is this result’s only artifact and is committed alongside this paper for that reason.

One naming hazard in `results/` is worth flagging directly rather than leaving for a reader to discover: `results/finetuned_track_b_token_level_metrics.json` is a byte-identical duplicate of `results/arm_b_metrics.json`, despite a filename that suggests otherwise (`docs/EXPERIMENT_LEDGER.md` E5 records how this arose). Neither this paper nor its source documents cite the duplicate; `arm_a_original_metrics.json` and `arm_b_metrics.json` are the two files this paper treats as canonical for the arm-a/arm-b comparison in §5.1.

References

- Belyi, M., Friel, R., Shao, S., & Sanyal, A. (2024/2025). Luna: An evaluation foundation model to catch language model hallucinations with high accuracy and low cost / Luna: A lightweight evaluation model to catch language model hallucinations with high accuracy and low cost. arXiv:2406.00975 (preprint) / *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track* (COLING 2025, camera-ready).
- Hu, X., Ru, D., Qiu, L., Guo, Q., Zhang, T., Xu, Y., Luo, Y., Liu, P., Zhang, Y., & Zhang, Z. (2024). RefChecker: Reference-based fine-grained hallucination checker and benchmark for large language models. arXiv:2405.14486.
- Kovács, Á., & Recski, G. (2025). LettuceDetect: A hallucination detection framework for RAG applications. arXiv:2502.17125.
- Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On faithfulness and factuality in abstractive summarization. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 1906–1919. <https://doi.org/10.18653/v1/2020.acl-main.173>

- Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., & Zhang, T. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 10862–10878. arXiv:2401.00396.
- Song, J., Wang, X., Zhu, J., Wu, Y., Cheng, X., Zhong, R., & Niu, C. (2024). RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 1548–1558. <https://doi.org/10.18653/v1/2024.emnlp-industry.113>
- Tang, L., Laban, P., & Durrett, G. (2024). MiniCheck: Efficient fact-checking of LLMs on grounding documents. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. <https://doi.org/10.18653/v1/2024.emnlp-main.499>
- Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Cooper, N., Adams, G., Howard, J., & Poli, I. (2024/2025). Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. arXiv:2412.13663. Also: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (2025).
- Zha, Y., Yang, Y., Li, R., & Hu, Z. (2023). AlignScore: Evaluating factual consistency with a unified alignment function. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. <https://doi.org/10.18653/v1/2023.acl-long.634>

Appendix A — Resource-constrained scaling replication

This appendix is methodologically separate from the paper’s argument and is not part of it. It reports a backbone-scaling comparison that shares infrastructure with the experiments above but tests nothing about `implicit_true`, contributes no evidence for or against §4, §5, or §6, and would be removed without affecting any claim the paper makes. It is included because the data exists and is worth reporting on its own terms, not because it supports anything else here.

What was run. A three-seed comparison between ModernBERT-base and ModernBERT-large under arm b’s recipe, intended as matched: the same seed should mean the same weight-initialization draw and data shuffle, so the backbone is the only difference. That intent is verifiable for two of the three seeds. This asymmetry sits entirely on the large-backbone side: the base arm itself is not in question. All three base seeds (42, 123, 456) have full, recorded hyperparameters and real, committed prediction dumps — the same three runs now serving as §5.2’s matched arm-b anchor and §5.3’s noise floor — so the verified-vs-unrecorded split below describes only which large-backbone seeds can be confirmed to match that base, not any gap in the base arm. Seeds 42 and 456 have full, recorded hyperparameters on both backbones and match on every axis (learning rate, epochs, checkpoint-selection metric, `implicit_true_weight`) other than backbone size — **2 verified matched seeds**. Seed 123’s large-backbone run was produced by the existing-model inference path rather than a training run’s own reporting; its metrics file records only `{"seed": 123}` under hyperparameters, so its recipe cannot be confirmed to match the base arm — **1 seed of unrecorded training configuration**, included in the counts below because it was run and its output is real, but not verified as matched. Deltas are computed per seed and then aggregated. The base arm of this scaling comparison is the same three training runs used as §5.3’s variance anchor for the training-side null — not a separate re-run, the identical runs read for a different purpose here.

Metric	Base mean	Large mean	Δ mean	Δ range	Direction	Published Δ (Kovács & Recski)
Response F1	0.7637	0.7948	+0.0311	+0.0292 ... +0.0323	3/3 seeds*	+0.0315
Span F1 (char-overlap)	0.5325	0.5733	+0.0408	+0.0379 ... +0.0446	3/3 seeds*	+0.0349

**2 of 3 seeds (42, 456) are verified recipe-matched across backbones; seed 123’s large-backbone run is of unrecorded training configuration (see “What was run,” above, and the caveats below). “3/3” is the observed direction across all three runs as executed, not a claim that all three are verified matched.*

At response level the observed gap is close to LettuceDetect’s published base→large gap (+0.0311 against +0.0315), so “replicates” is a fair description of that row. At span level the observed gap is **larger** than the published one (+0.0408 against +0.0349); the row does not merely reproduce the published difference, and we report the discrepancy rather than round it toward agreement. We do not know its cause and did not investigate it. Nothing in this comparison is a claim about either system’s quality as a method, and none of these figures is presented as a record.

No significance is claimed, and none is available at $n = 3$. The aggregation script emits raw per-seed values, means, min/max ranges, and per-seed paired deltas by construction, and no p-values. The reportable statement is that the direction was consistent across all 3 seeds run on both metrics, with the means and ranges above — on a base of 2 verified matched seeds (42, 456) plus one seed (123) of unrecorded training configuration.

The gain is recall-led. Response recall improves in 3/3 seeds (mean +0.0516), while response precision improves in only 2/3 (mean +0.0055, with one seed at −0.0028) and char-span precision likewise in 2/3 (mean +0.0217, one seed at −0.0207). Only recall is consistently better. The largest per-task movement is on Summary (+0.0931 mean F1, 3/3 seeds), against QA (+0.0375) and Data2txt (+0.0146).

What this is worth reporting for. LettuceDetect published its base→large scaling gap as a single point estimate with no variance information. This appendix supplies three matched seeds with means, ranges, and per-seed paired deltas for the same comparison, under materially more constrained compute: a single consumer GPU, and 4 epochs against their 6. The contribution, such as it is, is evidence about what a published scaling result looks like when re-run under constraint and with variance reported — not a statement about which system is better, which this design could not establish and does not attempt.

Caveats specific to this appendix, in addition to §8’s list:

- The base arm’s epoch cap is not uniform: seed 42 ran at 6 epochs, seeds 123 and 456 at 4. Seed 123 selected its final epoch and its ceiling under a longer cap is unknown. The base response-F1 spread across seeds is 0.0012 and the base→large gap is roughly 0.031, an order of magnitude larger, so this is unlikely to affect the direction — but the sweep is “seed, with a differing epoch cap on one base arm”, not “seed only”.
- Large seed 123’s metrics file records only its seed under hyperparameters and carries no validation block: it was produced by the existing-model inference path rather than by a training run’s own reporting, so that run’s cap and selected epoch are recorded nowhere.

- Per-row predictions were dumped for large seeds 42 and 456 but not for seed 123, so any stratified re-analysis of the large arm covers two of three seeds.
- The seed-42 large checkpoint is not published to the Hub.

Relationship to the rest of the paper: none. The scaling comparison and the annotation findings are independent results that happen to share a codebase and a benchmark. Neither corroborates the other, and this appendix should not be read as strengthening §4, §5, or §6 in any way. It is severable in full.